

Punteros

El capítulo anterior permitió construir una base: un programa en ejecución usa memoria, las variables ocupan posiciones y esas posiciones pueden representarse mediante direcciones. Ahora se estudia una herramienta propia de C que permite trabajar explícitamente con esas direcciones: los punteros.

En este capítulo no se estudiará todavía la gestión de memoria dinámica. No se usará `malloc`, `calloc`, `realloc` ni `free`. Esas funciones se dejan para el capítulo siguiente. Aquí el propósito es comprender con calma qué es un puntero, cómo se declara, cómo se relaciona con una variable, cómo se usan los operadores `&` y `*`, y por qué las direcciones permiten que una función modifique datos externos.

Idea central

Un puntero no es un tema aislado de sintaxis. Un puntero es una forma de razonar con variables, direcciones, funciones y datos almacenados en memoria.

1.1. Por qué los punteros importan

Valor y dirección: la primera separación

Cuando se empieza a programar en C, lo más natural es pensar que una variable es simplemente un nombre asociado a un valor. Por ejemplo, si se escribe:

```
int x = 25;
```

es común pensar únicamente en el número 25. Sin embargo, durante la ejecución del programa, ese valor debe guardarse en algún lugar de la memoria. Por tanto, la variable tiene dos aspectos importantes:

- el **valor** almacenado;
- la **dirección** donde ese valor se encuentra en memoria.

Esta separación es la puerta de entrada al estudio de los punteros. Mientras una variable normal permite trabajar cómodamente con un valor, un puntero permite guardar y usar una dirección.

Idea central

Estudiar punteros implica dejar de mirar solo el dato y empezar a mirar también dónde vive ese dato.

La Figura 1.1 presenta una primera representación conceptual. La variable `x` guarda el valor 7. El puntero `p`, en cambio, no guarda el valor 7; guarda la dirección donde se encuentra `x`. La dirección `0x1000` es simbólica y se usa solo para explicar la idea.

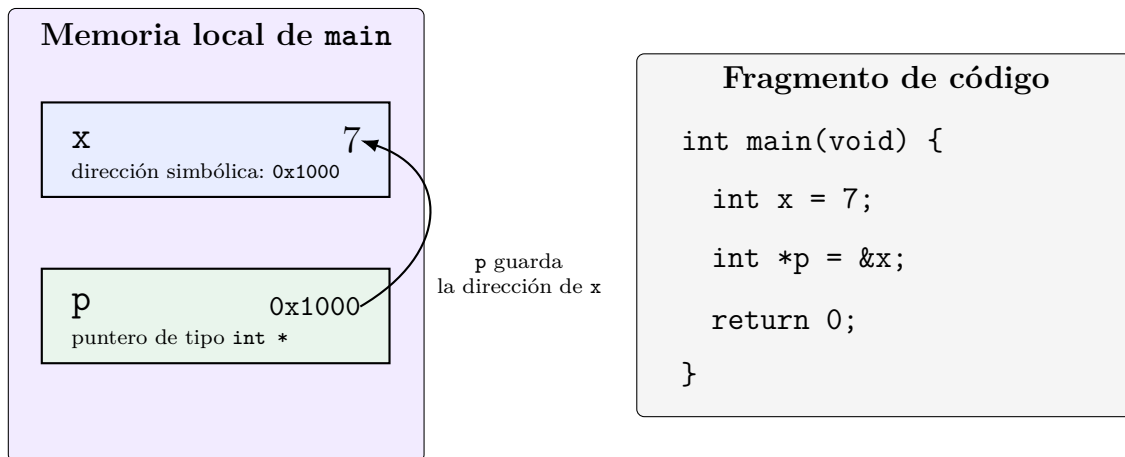


Figura 1.1: Relación conceptual entre una variable entera y un puntero a entero. La dirección `0x1000` es simbólica y se usa solo con fines explicativos.

En la Figura 1.1, la variable `x` tiene un valor directo: 7. El puntero `p` también es una variable, pero su contenido no es 7; su contenido es una dirección. Esa dirección corresponde, en el dibujo, al lugar donde vive `x`.

Clave de lectura

Un puntero es una variable cuyo contenido es una dirección de memoria. Dicho de otra forma, un puntero no guarda directamente el dato final que se desea trabajar, sino la ubicación donde ese dato se encuentra.

La utilidad práctica de los punteros

Los punteros se vuelven importantes porque permiten resolver problemas que serían incómodos o limitados si solo se usaran valores. Entre sus usos más frecuentes están:

- modificar variables originales desde una función;
- comunicar varios resultados hacia afuera de una función;
- trabajar directamente con direcciones de memoria;
- recorrer arreglos y cadenas de caracteres usando direcciones;

- preparar el camino para comprender la memoria dinámica.

Memoria

Por ahora, el énfasis estará en las direcciones de variables ya existentes y en el uso de punteros como parámetros de funciones. La reserva y liberación de memoria dinámica se estudiará después.

1.2. Un ejemplo mínimo: `x`, `&x`, `p` y `*p`

Antes de estudiar reglas más largas, conviene observar un programa mínimo. El siguiente código declara una variable entera, obtiene su dirección y guarda esa dirección en un puntero.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int main(void) { /* Funcion principal del programa. */
    int x = 25; /* Variable entera con valor inicial 25. */
    int *p = &x; /* p guarda la direccion de x. */

    printf("x = %d\n", x); /* Muestra el valor directo de x. */
    printf("&x = %p\n", (void *)&x); /* Muestra la direccion de x. */
    printf("p = %p\n", (void *)p); /* Muestra la direccion guardada en p. */
    printf("*p = %d\n", *p); /* Muestra el contenido ubicado en la direccion guardada en p. */

    return 0; /* Finalizacion exitosa del programa. */
}
```

Figura 1.2: Archivo ejemplo_01_x_direccion_puntero.c.

Al correr el respectivo código, la consola podría mostrar:

```
x = 25
&x = 0x7ffdf2a4c3bc
p = 0x7ffdf2a4c3bc
*p = 25
```

Nota sobre la salida

Las direcciones impresas con `%p` pueden cambiar en cada ejecución y en cada computador. Lo importante no es memorizar la dirección exacta, sino observar que `&x` y `p` muestran la misma dirección, porque `p` guarda la dirección de `x`.

El programa permite distinguir cuatro expresiones fundamentales:

Expresión	Interpretación
<code>x</code>	valor almacenado directamente en la variable.
<code>&x</code>	dirección de la variable <code>x</code> .
<code>p</code>	dirección guardada en el puntero.
<code>*p</code>	contenido de la dirección guardada en <code>p</code> .

Idea central

`&` pregunta: ¿dónde está?
`*` pregunta: ¿qué hay allá?

En esta salida, el valor de `x` es 25. La expresión `&x` muestra la dirección donde se encuentra almacenada la variable `x`. El valor mostrado por `p` coincide con `&x`, porque en la línea `int *p = &x;` se guardó en `p` la dirección de `x`. Finalmente, la expresión `*p` muestra el contenido que hay en la dirección almacenada en `p`; por eso también aparece el valor 25.

1.3. Declaración de punteros

Un puntero se declara indicando el tipo de dato al que puede apuntar. Por ejemplo:

```
int *p;
```

Esta declaración se lee como: `p` es un puntero a `int`. Eso significa que `p` puede guardar la dirección de una variable entera.

El tipo del puntero es importante porque le dice al compilador qué clase de dato se espera encontrar en la dirección apuntada. Un puntero de tipo `int *` está preparado para apuntar a un `int`; un puntero de tipo `double *` está preparado para apuntar a un `double`; y un puntero de tipo `char *` está preparado para apuntar a un `char`.

Declaración	Lectura
<code>int *p;</code>	<code>p</code> puede guardar la dirección de un entero.
<code>double *q;</code>	<code>q</code> puede guardar la dirección de un número real tipo <code>double</code> .
<code>char *c;</code>	<code>c</code> puede guardar la dirección de un carácter.

Cuidado

La declaración `int *p, q;` no declara dos punteros. Declara `p` como puntero a `int`, pero `q` como variable entera normal. Para declarar dos punteros, debe escribirse `int *p, *q;`.

1.4. Operador dirección: &

El operador `&` permite obtener la dirección de una variable. Si `x` es una variable entera, entonces `&x` representa la dirección donde vive `x` durante la ejecución del programa.

En el siguiente programa se usa de manera aislada el operador `&`. La intención es observar que una variable no solo tiene un valor, sino también una dirección en memoria.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int main(void) { /* Funcion principal del programa. */
    int x = 10; /* Variable entera con valor inicial 10. */

    printf("Valor de x = %d\n", x); /* Muestra el valor de x. */
    printf("Direccion de x = %p\n", (void *)&x); /* Muestra la direccion de x. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.3: Archivo ejemplo_02_operador_direccion.c.

Una salida posible sería:

```
Valor de x = 10
Direccion de x = 0x7ffd5c0a7d4c
```

La conversión `(void *)&x` se usa porque el especificador `%p` de `printf` espera recibir una dirección convertida a `void *`. Esta forma permite imprimir direcciones de manera adecuada en C.

1.5. Operador contenido: *

Cuando `*` se usa con un puntero ya declarado, permite acceder al contenido de la dirección guardada por ese puntero. Esta operación se llama **desreferenciación**. En palabras más sencillas: si `p` guarda una dirección, entonces `*p` permite entrar a esa dirección.

La salida esperada es:

```
x = 25
*p = 25
x despues = 99
```

La línea `*p = 99;` no cambia la dirección guardada en `p`. Lo que hace es ir a la dirección apuntada por `p` y escribir allí el valor 99. Como `p` apunta a `x`, el valor de `x` cambia.

Clave de lectura

Si `p` guarda la dirección de `x`, entonces `*p` permite leer o modificar el valor de `x` de manera indirecta.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int main(void) { /* Funcion principal del programa. */
    int x = 25; /* Variable entera con valor inicial 25. */
    int *p = &x; /* p guarda la direccion de x. */

    printf("x = %d\n", x); /* Muestra el valor directo de x. */
    printf("x = %d\n", *p); /* Muestra el valor encontrado en la direccion guardada en p. */

    *p = 99; /* Modifica x indirectamente mediante el puntero p. */

    printf("x despues = %d\n", x); /* Muestra x despues de la modificacion indirecta. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.4: Archivo ejemplo_03_operador_contenido.c.

1.6. El símbolo * en dos contextos

El símbolo * puede aparecer en una declaración o en una expresión. Es fundamental distinguir ambos casos.

Comparación

Declaración: `int *p;` significa que `p` es un puntero a entero.

Desreferenciación: `*p = 40;` significa entrar a la dirección guardada en `p` y escribir allí.

El siguiente ejemplo reúne los dos usos principales del asterisco en este tema. Primero aparece en la declaración de un puntero; luego aparece en una expresión para acceder al contenido apuntado.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int main(void) { /* Funcion principal del programa. */
    int x = 5; /* Variable entera con valor inicial 5. */

    int *p = &x; /* Declaracion de puntero: aqui el * forma parte de la declaracion. */

    *p = 40; /* Desreferenciacion: aqui el * accede al contenido apuntado por p. */

    printf("x = %d\n", x); /* Muestra el valor final de x. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.5: Archivo ejemplo_04_dos_usos_asterisco.c.

La salida esperada es:

```
x = 40
```

Cuidado

El mismo símbolo puede tener significados diferentes según el contexto. En `int *p`; ayuda a declarar un puntero. En `*p = 40`; permite entrar a la dirección guardada en `p`.

1.7. Paso por valor y paso de direcciones

En C, los argumentos de una función se pasan por valor. Esto significa que la función recibe copias de los argumentos usados en la llamada [2, 3]. Si se envía una variable entera, la función recibe una copia del entero. Si se envía una dirección, la función recibe una copia de esa dirección.

La segunda situación es la que vuelve útiles a los punteros: aunque la dirección recibida por la función sea una copia, esa copia todavía permite llegar a la variable original.

Idea central

Si se envía un entero, la función recibe una copia del entero. Si se envía una dirección, la función recibe una copia de la dirección.

Intento de cambio sin punteros

Para reforzar la diferencia entre valores y direcciones, el programa en la Figura 1.6 intenta modificar una variable desde una función, pero envía solamente el valor. Por eso la variable original no cambia.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

void cambiar(int n) { /* n recibe una copia del valor enviado desde main. */
    n = 100; /* Se modifica la copia local, no la variable original. */

    printf("Dentro de cambiar: n = %d\n", n); /* Muestra la copia modificada. */
}

int main(void) { /* Funcion principal del programa. */
    int x = 5; /* Variable original. */

    printf("Antes: x = %d\n", x); /* Muestra x antes de llamar la funcion. */

    cambiar(x); /* Se envia el valor de x. La funcion recibe una copia. */

    printf("Despues: x = %d\n", x); /* x no cambia porque solo se modifiko la copia. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.6: Archivo ejemplo_05_cambio_sin_punteros.c.

La salida esperada es:

```
Antes: x = 5
Dentro de cambiar: n = 100
Despues: x = 5
```

La función `cambiar` modifica su variable local `n`, pero no modifica `x`. Esto ocurre porque `n` es una copia del valor de `x`, no la variable original.

Cambio usando una dirección

Ahora se modifica el mismo planteamiento enviando la dirección de la variable. En el siguiente código, la función recibe un puntero y escribe directamente en la variable original.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */
#include <stddef.h> /* Incluye la definicion de NULL. */

void cambiar(int *p) { /* p recibe una copia de una direccion. */
    if (p == NULL) { /* Verifica si la direccion recibida no es valida. */
        return; /* Sale de la funcion para evitar usar un puntero invalido. */
    }

    *p = 100; /* Escribe 100 en la variable ubicada en la direccion recibida. */
}

int main(void) { /* Funcion principal del programa. */
    int x = 5; /* Variable original. */

    printf("Antes: x = %d\n", x); /* Muestra x antes de llamar la funcion. */

    cambiar(&x); /* Envia la direccion de x. */

    printf("Despues: x = %d\n", x); /* x cambia porque la funcion escribio en su
    direccion. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.7: Archivo `ejemplo_06_cambio_con_direccion.c`.

La salida esperada es:

```
Antes: x = 5
Despues: x = 100
```

En este caso, `p` también es una variable local de la función `cambiar`; sin embargo, su contenido es una dirección. Aunque esa dirección llega como copia, permite acceder a `x`. Por eso la instrucción `*p = 100;` modifica la variable original.

Clave de lectura

En C no se pasa una variable completa por referencia como ocurre en otros lenguajes. Se pasa una copia. Pero si esa copia es una dirección, la función puede usarla para llegar al dato original.

1.8. Intercambio de dos variables usando punteros

Intercambiar dos variables desde una función es uno de los ejemplos más claros para justificar los punteros. Primero se muestra el intento que no funciona y luego la versión correcta usando direcciones.

Intento de intercambio sin punteros

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

void intercambiar(int a, int b) { /* a y b son copias locales. */
    int temp = a; /* Guarda temporalmente el valor de a. */

    a = b; /* Cambia la copia a. */
    b = temp; /* Cambia la copia b. */

    printf("Dentro de intercambiar: a = %d, b = %d\n", a, b); /* Muestra las copias. */
}

int main(void) { /* Funcion principal del programa. */
    int x = 10; /* Primera variable original. */
    int y = 20; /* Segunda variable original. */

    printf("Antes: x = %d, y = %d\n", x, y); /* Valores antes del intercambio. */

    intercambiar(x, y); /* Envia valores; la funcion recibe copias. */

    printf("Despues: x = %d, y = %d\n", x, y); /* Las variables originales no cambian. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.8: Archivo ejemplo_07_intercambio_sin_punteros.c.

La salida esperada es:

```
Antes: x = 10, y = 20
Dentro de intercambiar: a = 20, b = 10
Despues: x = 10, y = 20
```

Cuidado

La función `intercambiar` parece funcionar dentro de su propio espacio local, pero no cambia las variables originales `x` y `y`. Solo intercambia las copias `a` y `b`.

Intercambio correcto con punteros

Para intercambiar las variables originales, la función debe recibir sus direcciones. Así puede entrar a las posiciones de memoria donde viven `x` y `y`.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */
#include <stddef.h> /* Incluye la definicion de NULL. */

void intercambiar(int *a, int *b) { /* a y b reciben direcciones de variables enteras. */
    if (a == NULL || b == NULL) { /* Verifica si alguna direccion no es valida. */
        return; /* Sale de la funcion sin hacer cambios. */
    }

    int temp = *a; /* Guarda el contenido apuntado por a. */
    *a = *b; /* Escribe en la direccion apuntada por a el contenido apuntado por b. */
    *b = temp; /* Escribe en la direccion apuntada por b el valor temporal. */
}

int main(void) { /* Funcion principal del programa. */
    int x = 10; /* Primera variable original. */
    int y = 20; /* Segunda variable original. */

    printf("Antes: x = %d, y = %d\n", x, y); /* Valores antes del intercambio. */

    intercambiar(&x, &y); /* Envia las direcciones de x y y. */

    printf("Despues: x = %d, y = %d\n", x, y); /* Ahora las variables originales
    cambiaron. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.9: Archivo `ejemplo_08_intercambio_con_punteros.c`.

La salida esperada es:

```
Antes: x = 10, y = 20
Despues: x = 20, y = 10
```

La diferencia está en la llamada:

```
intercambiar(&x, &y);
```

Allí no se envían los valores 10 y 20; se envían las direcciones de `x` y `y`. Dentro de la función, `a` y `b` reciben copias de esas direcciones. Al usar `*a` y `*b`, la función puede leer y escribir sobre las variables originales.

1.9. Punteros como canales de salida

En C, una función puede devolver directamente un solo valor mediante `return`. Esto es suficiente en muchos casos. Por ejemplo, una función que suma dos números puede retornar el resultado de la suma.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int sumar(int a, int b) { /* Recibe dos enteros como entrada. */
    return a + b; /* Devuelve directamente un solo resultado. */
}

int main(void) { /* Funcion principal del programa. */
    int resultado = sumar(4, 7); /* Llama la funcion y guarda el valor retornado. */

    printf("Resultado = %d\n", resultado); /* Muestra el resultado. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.10: Archivo ejemplo_09_funcion_return.c.

La salida esperada es:

```
Resultado = 11
```

Sin embargo, muchos problemas requieren producir varios datos al mismo tiempo. Por ejemplo, a partir de tres números puede interesar calcular:

- la suma;
- el valor mayor;
- el valor menor.

Crear una función separada para cada resultado puede funcionar, pero a veces genera código repetitivo. Los punteros permiten una alternativa útil: una función puede recibir direcciones y escribir los resultados directamente en variables externas.

En la Figura 1.11 se presenta una función que calcula varios resultados en una sola llamada. Para lograrlo, algunos parámetros se usan como entradas normales y otros se usan como direcciones donde la función escribirá las respuestas.

La función `analizarNumeros` recibe tres valores de entrada: `a`, `b` y `c`. Además, recibe tres direcciones: `suma`, `mayor` y `menor`. En esas direcciones la función escribe los resultados calculados, de manera que las variables declaradas en `main` quedan actualizadas después de la llamada.

Al ejecutar el programa mostrado en la Figura 1.11, la salida esperada es:

```
Suma   = 26
Mayor  = 15
Menor  = 3
```

```

#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */
#include <stddef.h> /* Incluye la definicion de NULL. */

void analizarNumeros(int a, int b, int c, int *suma, int *mayor, int *menor) {
    if (suma == NULL || mayor == NULL || menor == NULL) { /* Verifica direcciones. */
        return; /* Sale si alguna direccion no es valida. */
    }

    *suma = a + b + c; /* Escribe la suma en la direccion recibida. */

    *mayor = a; /* Supone inicialmente que a es el mayor. */
    if (b > *mayor) { /* Compara b con el mayor actual. */
        *mayor = b; /* Actualiza el mayor si b es mas grande. */
    }
    if (c > *mayor) { /* Compara c con el mayor actual. */
        *mayor = c; /* Actualiza el mayor si c es mas grande. */
    }

    *menor = a; /* Supone inicialmente que a es el menor. */
    if (b < *menor) { /* Compara b con el menor actual. */
        *menor = b; /* Actualiza el menor si b es mas pequeno. */
    }
    if (c < *menor) { /* Compara c con el menor actual. */
        *menor = c; /* Actualiza el menor si c es mas pequeno. */
    }
}

int main(void) { /* Funcion principal del programa. */
    int x = 8; /* Primer dato de entrada. */
    int y = 3; /* Segundo dato de entrada. */
    int z = 15; /* Tercer dato de entrada. */

    int suma; /* Variable donde se escribira la suma. */
    int mayor; /* Variable donde se escribira el mayor. */
    int menor; /* Variable donde se escribira el menor. */

    analizarNumeros(x, y, z, &suma, &mayor, &menor); /* Envia valores y direcciones. */

    printf("Suma = %d\n", suma); /* Muestra la suma. */
    printf("Mayor = %d\n", mayor); /* Muestra el mayor. */
    printf("Menor = %d\n", menor); /* Muestra el menor. */

    return 0; /* Finalizacion exitosa. */
}

```

Figura 1.11: Archivo ejemplo_10_varios_resultados_punteros.c.

La función `analizarNumeros` no devuelve tres valores mediante `return`. En cambio, recibe tres direcciones:

- `int *suma`: dirección donde se debe escribir la suma;
- `int *mayor`: dirección donde se debe escribir el mayor valor;

- `int *menor`: dirección donde se debe escribir el menor valor.

Cuando en `main` se llama:

```
analizarNumeros(x, y, z, &suma, &mayor, &menor);
```

los tres primeros argumentos se envían como valores. En cambio, los últimos tres argumentos se envían como direcciones. Por eso la función puede escribir los resultados en variables que pertenecen a `main`.

Idea central

Un puntero usado como parámetro puede actuar como canal de salida: la función recibe una dirección y escribe allí un resultado.

1.10. Punteros, arreglos y cadenas: una primera conexión

Aunque el estudio detallado de arreglos y cadenas puede hacerse en otro capítulo, conviene mencionar desde ahora una conexión importante: en C, los arreglos y las cadenas de caracteres están muy relacionados con direcciones. El nombre de un arreglo puede usarse, en muchos contextos, como una dirección hacia su primer elemento.

El siguiente ejemplo muestra una primera relación entre un arreglo, su primer elemento y un puntero.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */

int main(void) { /* Funcion principal del programa. */
    int datos[3] = {10, 20, 30}; /* Arreglo de tres enteros. */
    int *p = datos; /* p apunta al primer elemento del arreglo. */

    printf("datos[0] = %d\n", datos[0]); /* Acceso mediante indice. */
    printf("*p      = %d\n", *p); /* Acceso mediante puntero. */
    printf("*(p + 1) = %d\n", *(p + 1)); /* Acceso al segundo elemento. */
    printf("*(p + 2) = %d\n", *(p + 2)); /* Acceso al tercer elemento. */

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.12: Archivo `ejemplo_11_puntero_y_arreglo_basico.c`.

La salida esperada es:

```
datos[0] = 10
*p      = 10
*(p + 1) = 20
*(p + 2) = 30
```

Clave de lectura

En muchos contextos, el nombre de un arreglo se comporta como la dirección de su primer elemento. Por eso `datos` puede asignarse a un puntero `int *`. Esta idea será importante cuando se estudien arreglos, cadenas y memoria dinámica con más detalle.

1.11. Buenas prácticas iniciales con punteros

Antes de avanzar hacia gestión de memoria, conviene fijar algunas prácticas básicas. Estas reglas no sustituyen el razonamiento, pero ayudan a evitar errores comunes.

- Inicializar los punteros cuando se declaran.
- Usar `NULL` cuando un puntero no apunta todavía a una dirección válida.
- Verificar un puntero antes de desreferenciarlo si existe la posibilidad de que sea `NULL`.
- No usar `*p` si no se sabe con claridad hacia dónde apunta `p`.
- Recordar que una dirección copiada sigue permitiendo llegar al dato original.

```
#include <stdio.h> /* Incluye la biblioteca necesaria para printf. */
#include <stddef.h> /* Incluye la definicion de NULL. */

int main(void) { /* Funcion principal del programa. */
    int *p = NULL; /* El puntero se inicializa en NULL porque aun no apunta a un int. */

    if (p != NULL) { /* Solo se desreferencia si apunta a una direccion valida. */
        printf("Valor = %d\n", *p);
    } else {
        printf("p no apunta a una direccion valida.\n");
    }

    return 0; /* Finalizacion exitosa. */
}
```

Figura 1.13: Archivo ejemplo_12_inicializar_puntero_NULL.c.

La salida esperada es:

```
p no apunta a una direccion valida.
```

Cuidado

Un puntero sin inicializar contiene un valor indeterminado. Desreferenciarlo puede producir errores graves. Antes de usar `*p`, debe existir claridad sobre la dirección guardada en `p`.

1.12. Práctica integradora de punteros en C

La siguiente práctica integra varias ideas del capítulo en un solo programa: direcciones, desreferenciación, modificación indirecta, intercambio de valores y uso de punteros como canales de salida.

```
#include <stdio.h> /* Incluye printf y scanf. */
#include <stddef.h> /* Incluye la definicion de NULL. */

void duplicar(int *p) { /* Recibe la direccion de un entero. */
    if (p != NULL) { /* Verifica que el puntero sea valido. */
        *p = (*p) * 2; /* Duplica el valor almacenado en la direccion apuntada. */
    }
}

void intercambiar(int *a, int *b) { /* Recibe dos direcciones de enteros. */
    if (a == NULL || b == NULL) { /* Verifica que ambas direcciones sean validas. */
        return; /* Sale si alguna direccion no es valida. */
    }

    int temp = *a; /* Guarda temporalmente el contenido apuntado por a. */
    *a = *b; /* Copia en a el contenido apuntado por b. */
    *b = temp; /* Copia en b el valor temporal. */
}
```

Figura 1.14: Archivo practica_01_menu_punteros_c.c (parte 1 de 4).

Una posible interacción de consola sería:

```
===== MENU CORTO DE PUNTEROS =====
1. Mostrar x, &x, px y *px
2. Duplicar x usando el puntero px
3. Intercambiar x y y usando punteros
4. Analizar x, y, z usando punteros de salida
0. Salir
Seleccione una opcion: 1
x   = 10
&x  = 0x7ffd9b2c4a1c
px   = 0x7ffd9b2c4a1c
*px  = 10
```

Para estudiar con calma

Esta práctica muestra una misma idea desde varios ángulos: un puntero puede leer una variable, modificarla indirectamente, permitir el intercambio de dos variables y servir como canal de salida para varios resultados.

```

void analizarNumeros(int a, int b, int c, int *suma, int *mayor, int *menor) {
    if (suma == NULL || mayor == NULL || menor == NULL) { /* Verifica direcciones. */
        return; /* Sale si alguna direccion no es valida. */
    }

    *suma = a + b + c; /* Escribe la suma en la direccion recibida. */

    *mayor = a; /* Supone inicialmente que a es el mayor. */
    if (b > *mayor) { /* Compara b. */
        *mayor = b; /* Actualiza el mayor. */
    }
    if (c > *mayor) { /* Compara c. */
        *mayor = c; /* Actualiza el mayor. */
    }

    *menor = a; /* Supone inicialmente que a es el menor. */
    if (b < *menor) { /* Compara b. */
        *menor = b; /* Actualiza el menor. */
    }
    if (c < *menor) { /* Compara c. */
        *menor = c; /* Actualiza el menor. */
    }
}

```

Figura 1.15: Archivo practica_01_menu_punteros_c.c (parte 2 de 4).

```

int main(void) { /* Funcion principal del programa. */
    int x = 10; /* Primera variable de trabajo. */
    int y = 20; /* Segunda variable de trabajo. */
    int z = 5; /* Tercera variable de trabajo. */

    int *px = &x; /* px guarda la direccion de x. */

    int suma; /* Variable para recibir la suma. */
    int mayor; /* Variable para recibir el mayor. */
    int menor; /* Variable para recibir el menor. */
    int opcion; /* Opcion seleccionada por el usuario. */

    do { /* Ciclo principal del menu. */
        printf("\n===== MENU CORTO DE PUNTEROS =====\n");
        printf("1. Mostrar x, &x, px y *px\n");
        printf("2. Duplicar x usando el puntero px\n");
        printf("3. Intercambiar x y y usando punteros\n");
        printf("4. Analizar x, y, z usando punteros de salida\n");
        printf("0. Salir\n");
        printf("Seleccione una opcion: ");
        scanf("%d", &opcion); /* Lee la opcion. */
    }
}

```

Figura 1.16: Archivo practica_01_menu_punteros_c.c (parte 3 de 4).


```
switch (opcion) { /* Decide que accion ejecutar. */
    case 1:
        printf("x    = %d\n", x);
        printf("&x  = %p\n", (void *)&x);
        printf("px   = %p\n", (void *)px);
        printf("*px  = %d\n", *px);
        break;

    case 2:
        printf("Antes: x = %d\n", x);
        duplicar(px); /* Modifica x usando su direccion. */
        printf("Despues: x = %d\n", x);
        break;

    case 3:
        printf("Antes: x = %d, y = %d\n", x, y);
        intercambiar(&x, &y); /* Envia direcciones de x y y. */
        printf("Despues: x = %d, y = %d\n", x, y);
        break;

    case 4:
        analizarNumeros(x, y, z, &suma, &mayor, &menor);
        printf("Suma    = %d\n", suma);
        printf("Mayor   = %d\n", mayor);
        printf("Menor   = %d\n", menor);
        break;

    case 0:
        printf("Fin del programa.\n");
        break;

    default:
        printf("Opcion no valida.\n");
        break;
}
} while (opcion != 0); /* Repite mientras no se seleccione salir. */

return 0; /* Finalizacion exitosa. */
}
```

Figura 1.17: Archivo practica_01_menu_punteros_c.c (parte 4 de 4).

1.13. Cierre del capítulo

En este capítulo se estudió el uso básico de punteros en C. La idea central fue distinguir entre valor, dirección y contenido apuntado. También se observó que una función recibe copias de sus argumentos, pero que una copia de una dirección permite llegar al dato original.

Con esta base, el siguiente capítulo puede abordar la gestión de memoria. Allí se estudiará cómo solicitar memoria durante la ejecución, cómo liberarla correctamente y qué errores aparecen cuando no se administra bien la memoria dinámica.

Idea central

Un puntero permite que una parte del programa trabaje con la ubicación de un dato, no solo con una copia de su valor.

Referencias

Bibliografía

- [1] International Organization for Standardization. *ISO/IEC 9899:2011, Information technology – Programming languages – C*. ISO, 2011. Página oficial: <https://www.iso.org/standard/57853.html>. También se consulta el borrador público N1570 del comité ISO/IEC JTC1/SC22/WG14: <https://www.iso-9899.info/n1570.html>.
- [2] Kernighan, B. W., y Ritchie, D. M. *The C Programming Language*. Segunda edición. Prentice Hall, 1988.
- [3] Harbison, S. P., y Steele, G. L. *C: A Reference Manual*. Quinta edición. Prentice Hall, 2002.